

Federico Ceragioli

📍 Yokohama, Japan | 🔗 <https://github.com/fceragio> | 💼 <https://www.linkedin.com/in/federico-ceragioli-445386358/>

Profile

Student at 42 Tokyo, a peer-to-peer programming school with a self-directed, project-based curriculum. Experienced in full-stack and backend development through team and solo academic projects. Comfortable working autonomously, delivering under peer review, and communicating in writing across multilingual teams. Motivated to contribute to real client projects and grow in a hands-on environment.

Education

42 Tokyo — Peer-to-Peer Programming School Completed a project-based curriculum covering systems programming, algorithms, C/C++, full-stack web development, containerization, and collaborative workflows.

Technical Skills

Languages: JavaScript, TypeScript, C, C++ **Frameworks & Libraries:** NestJS, React (basics) **Infrastructure & Tools:** Docker, Docker Compose, Git, Nginx, PostgreSQL **Code Quality:** husky, prettier, eslint **Other:** Unix systems, multithreading, memory management, Object oriented programming

Soft Skills

- Clear written and verbal communication
 - Effective teamwork in peer-review environments
 - Problem-solving mindset with attention to detail
 - Adaptability and eagerness to learn
-

Projects

ft_transcendence (TypeScript, NestJS, React, PostgreSQL, Docker) Full-stack multiplayer Pong application built as a team project. Responsible for core game logic: WebSocket-based real-time state management, player reconnection and disconnect handling, AI opponent behavior, ball physics (reflection angle calculation based on hit position), and frontend cheat detection (paddle position validation). Also maintained code quality via husky, prettier, and eslint. → <https://github.com/fceragio> (alex216 organization)

Inception (Dockerfile, Docker Compose) Designed and deployed a containerized infrastructure from scratch: Nginx reverse proxy, WordPress, and MariaDB — each in its own container.

philosophers (C) Dining philosophers problem: multithreaded simulation using mutexes and thread synchronization.

ft_printf / get_next_line / libft (C) Reimplementation of standard C library functions: custom printf, line reader, and utility library.

C++ Modules 00-09 (C++) Series of exercises covering object-oriented programming, operator overloading, inheritance, polymorphism, templates, and exception handling.

Languages

- **Japanese:** Conversational
- **English:** Fluent
- **Italian:** Native